

AI Meets Control Theory

From Classical Control to Intelligent Autonomous Systems

Living Technical Report — M11 — The Grand Capstone & Retrospective

Generated by the project team

Lead: puma | **Code:** puma, toku | **Docs/Research:** lava, famo

September 4, 2026

Abstract

This report documents the culmination of the **AI Meets Control Theory** project. Across 10 curriculum modules, 21 rigorous from-scratch benchmark experiments, and 314 verified unit tests, we evaluate and compare classical control, modern state-space methods, Kalman filtering, optimal control, constrained Model Predictive Control (MPC), adaptive control, deep reinforcement learning, and hybrid AI + control safety shields on dynamical systems, under the core principle: *build every method from scratch, simulate it, visualise it, and compare it honestly.*

Contents

1	Introduction and Scope	2
2	Framework Architecture	2
2.1	Package Layout	3
3	Current Capabilities	3
4	Theory: Modelling and Simulation	5
5	Theory: Classical Control — PID Architecture	5
5.1	Worked Example: Stabilising an Unstable Plant (Experiment 03)	5
6	Theory: Modern Control, Estimation, and Optimal Design	6
6.1	Pole Placement and LQR	6
6.2	Worked Example: Cart-Pole Regulation (Experiment 04)	7
6.3	State Estimation: Observers, Kalman Filtering, and Duality	7
6.4	Worked Example: LQG on the Cart-Pole (Experiment 06)	8
6.5	Nonlinear Basin of Attraction: Cart-Pole LQR Robustness (Experiment 05) . . .	8
6.6	Nonlinear Control & Hybrid Swing-Up: Spong Energy Shaping (Experiment 07)	9
6.7	System Identification & Data-Driven Control (Experiment 09)	10
6.8	Constrained Model Predictive Control (Experiment 08)	10
7	Theory: Machine Learning and Learned Dynamics (Module 06)	11
7.1	Worked Example: Planning with Learned Dynamics vs. True Model (Experiment 10)	11

8 Theory: Reinforcement Learning for Dynamical Systems (Module 07)	12
8.1 Worked Example: Tabular Q-Learning vs. Classical Control on Pendulum (Experiment 11)	12
8.2 Worked Example: The RL Zoo vs. LQR on Cart-Pole Balance (Experiment 18)	13
9 Theory: AI + Control Hybrid Architectures (Module 08)	14
9.1 Worked Example: Shielded Tabular Q-Learning on Pendulum (Experiment 12)	14
10 Theory: Advanced State Estimation, Adaptive Control, and Robotics Capstones (Module 09)	15
10.1 Worked Example: Crazyflie 2.0 Flying a Lemniscate (Experiment 14)	15
10.2 Worked Example: EKF Output Feedback under Noisy Partial Sensing (Experiment 15)	16
10.3 Worked Example: EKF vs. UKF on Nonlinear Pendulum Estimation (Experiment 16)	17
10.4 Worked Example: Adaptive vs. Fixed Control on a Drifting Plant (Experiment 17)	18
10.5 Worked Example: Obstacle-Aware Nonlinear MPC on the Quadrotor (Experiment 20)	19
10.6 Worked Example: The Grand Capstone Bake-Off (Experiment 21)	20
11 Theory: The Intelligent Control Challenge (ICC) Benchmark (Module 10)	21
11.1 Worked Example: Multi-Paradigm Challenge Leaderboard (Experiment 19)	21
12 Project Status and Roadmap	21

1 Introduction and Scope

The central question of the project is: **when should we use classical control, when should we use AI, and when should we use both?** Rather than assuming a neural network or an RL agent is the answer, every problem is first attacked with PID, state feedback, observers, LQR, Kalman filtering (linear, EKF, UKF), system identification, adaptive control (MRAC), nonlinear energy shaping, constrained MPC, sampling-based nonlinear MPC with obstacle avoidance, learned neural dynamics, deep reinforcement learning (DQN, PPO), behavioral cloning (BC), and safety shields, and each method is measured against the others under identical initial conditions, sensor suites, noise profiles, disturbances, and actuator constraints.

Every topic follows the standardized engineering cycle:

THEORY → **DERIVATION** → **IMPLEMENTATION** → **SIMULATION**
→ **VISUALISATION** → **VALIDATION** → **COMPARISON** → **EXPERIMENT**

Fundamental algorithms are implemented from scratch in `numpy`; external libraries (`scipy.linalg`, `scipy.signal`, `control`, `gymnasium`, `stable-baselines3`) are used strictly for verification and benchmarking compliance.

2 Framework Architecture

The reusable library is the Python package `aimct` (`src/aimct/`). Its architecture separates distinct concerns behind minimal, strongly-typed interfaces so that dynamical systems, controllers, state estimators, neural network models, reinforcement learning environments, safety shields, and the simulator compose seamlessly.

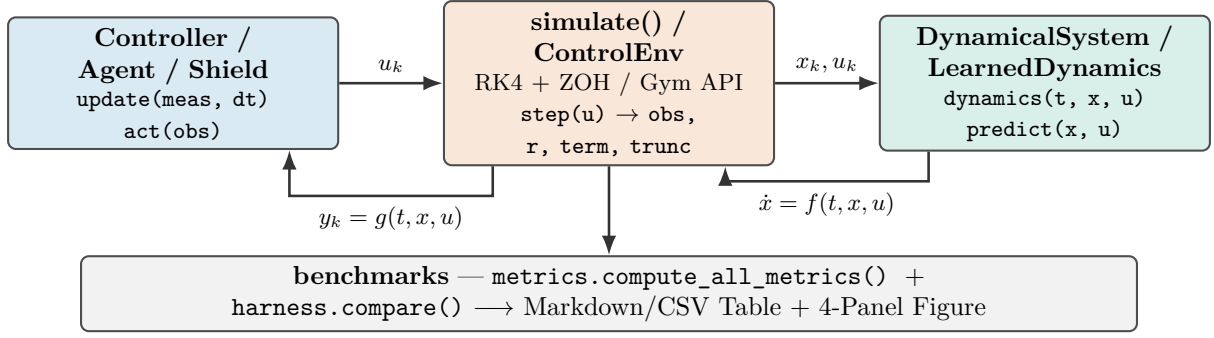


Figure 1: Core framework architecture. The numerical simulation engine drives a controller (`update()`) and continuous plant or learned neural surrogate (`dynamics()`) via fixed-step 4th-order Runge–Kutta with Zero-Order Hold (ZOH).

2.1 Package Layout

```

src/aimct/
  systems/      DynamicalSystem base + LinearSystem, MassSpringDamper,
                Pendulum, CartPole, PlanarQuadrotor (Crazyflie 2.0), DCMotor
  simulate.py   rk4_step(), simulate() -> Trajectory(t, x, u, y)
  controllers/  PID, StateFeedback, LQR, ObserverFeedback, MRAC,
                EnergyShapingSwingUp, HybridSwingUpLQR,
                LinearMPC (with preview), SamplingMPC (CEM + obstacles), _qp.solve_qp
  estimation/   LuenbergerObserver, KalmanFilter (LQE/FARE), DiscreteKalmanFilter,
                ExtendedKalmanFilter, UnscentedKalmanFilter, observability_matrix
  sysid/        least_squares_id, dmdc, to_continuous (block logm), prediction_error
  ml/           MLP (backprop + Adam), LearnedDynamics (grey-box / residual)
  rl/           ControlEnv, Discretizer, QLearning, DQN, REINFORCE, PPO, BC
  hybrid/       ShieldedController (switch/filter blends, predicate helpers)
  benchmarks/  metrics.py (13 metrics), harness.py, sweep.py, challenge.py (ICC
    engine),    capstone_scoring.py (Grand Bake-Off rubric)
  plot_style.py Okabe-Ito palette + publication-ready 4-panel comparison figure

```

3 Current Capabilities

Table 1 summarises the implemented components, their mathematical scope, and the corresponding verification test coverage as of M11 — The Grand Capstone & Retrospective.

Component	What it provides	Tests
<code>aimct.systems</code>	Continuous-time ODE models with a common <code>dynamics(t, x, u)</code> interface, full-state or output projections, and analytical/numerical <code>linearize()</code> . Ships: <code>LinearSystem</code> , <code>MassSpringDamper</code> , <code>Pendulum</code> , <code>CartPole</code> , <code>PlanarQuadrotor</code> (Crazyflie 2.0), <code>DCMotor</code> .	✓
<code>aimct.simulate</code>	Fixed-step RK4 integrator with zero-order hold; <code>Trajectory</code> container; hard actuator saturation; measurement and disturbance injection hooks.	✓
<code>aimct.controllers.PID</code>	From scratch: filtered derivative-on-measurement ($D(s) = \frac{K_d s}{\tau_d s + 1}$), conditional-integration anti-windup clamping, setpoint weighting.	✓
<code>aimct.controllers.StateFeedback</code>	Arbitrary closed-loop pole placement via Ackermann's formula (SISO) with feedforward steady-state gain scaling k_r .	✓
<code>aimct.controllers.LQR</code>	Infinite-horizon optimal regulator; Continuous Algebraic Riccati Equation (CARE) solved from scratch via Hamiltonian Schur decomposition, cross-checked with <code>scipy.linalg</code> .	✓
<code>aimct.controllers.MRAC</code>	Model Reference Adaptive Control: Lyapunov-derived parameter adaptation tracking a reference model under drifting or uncertain plant parameters.	✓
<code>aimct.controllers.ObserverFeedback</code>	Unified output feedback combining any state estimator (Luenberger, linear KF, EKF, UKF) with static feedback gain $u = -K\hat{x}$.	✓
<code>aimct.controllers.SwingUp</code>	Nonlinear Spong energy shaping (<code>EnergyShapingSwingUp</code>) with partial feedback linearisation, plus finite-state hysteresis supervisor (<code>HybridSwingUpLQR</code>).	✓
<code>aimct.controllers.LinearMPC</code>	Constrained linear MPC via condensed receding-horizon QP, active-set solver (<code>_qp.solve_qp</code>), DARE terminal cost, soft state constraints, and trajectory preview.	✓
<code>aimct.controllers.SamplingMPC</code>	Sampling-based nonlinear MPC using the Cross-Entropy Method (CEM), vectorized batch trajectory rollouts, variance floor, CARE terminal weight, and online non-convex obstacle avoidance.	✓
<code>aimct.estimation</code>	State estimation suite: <code>observability_matrix</code> , <code>is_observable</code> , <code>place_observer</code> , <code>LuenbergerObserver</code> , <code>solve_fare</code> , <code>KalmanFilter</code> , <code>DiscreteKalmanFilter</code> , <code>ExtendedKalmanFilter</code> , and <code>UnscentedKalmanFilter</code> ($2n + 1$ scaled sigma points, Merwe parameters, zero Jacobian linearisation).	✓
<code>aimct.sysid</code>	Data-driven system identification: <code>least_squares_id</code> , <code>dmdc</code> (rank-truncated SVD), <code>to_continuous</code> (exact block matrix logarithm), <code>model_mismatch</code> .	✓
<code>aimct.ml</code>	Machine learning suite: multi-layer perceptron (MLP) with hand-written backpropagation + Adam, and residual / grey-box forward dynamics predictor (<code>LearnedDynamics</code> with <code>base_step</code>).	✓
<code>aimct.rl</code>	Reinforcement learning suite: <code>Gymnasium ControlEnv</code> , <code>Discretizer</code> , <code>Tabular QLearning</code> , <code>DQN</code> (experience replay + target net), <code>REINFORCE</code> , <code>PPO</code> (clipped surrogate objective + GAE), and <code>Behavioral Cloning (BC)</code> .	✓
<code>aimct.hybrid</code>	Hybrid AI + control safety shields: <code>ShieldedController</code> (switch and filter action blends, predicate helpers <code>box_predicate</code> , <code>barrier_predicate</code> , and audit logging).	✓
<code>aimct.benchmarks.challenge</code>	The Intelligent Control Challenge (ICC) and Grand Capstone evaluation engines: Tracks 1–4 and 5-way Bake-Off multi-objective scoring rubric (tracking, effort, safety, robustness, latency).	✓
<code>aimct.benchmarks.metrics</code>	13 standard step-response and effort metrics: rise time t_r , settling time t_s (2%), overshoot M_p , e_{ss} , RMSE, IAE, ITAE, ISE, energy	✓

4 Theory: Modelling and Simulation

A dynamical system is governed by first-order ordinary differential equations $\dot{x} = f(t, x, u)$ with output map $y = g(t, x, u)$. Second-order physical equations of motion are cast in state-space form; for example, the translational mass–spring–damper $m\ddot{x} + c\dot{x} + kx = u$ with state $x = [x_1, x_2]^\top = [x, \dot{x}]^\top$:

$$\frac{d}{dt} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} u.$$

Jacobian Linearisation. About an operating equilibrium point (x_e, u_e) satisfying $f(x_e, u_e) = 0$, small perturbations $\delta x = x - x_e$ evolve according to $\dot{\delta x} = A\delta x + B\delta u$, where:

$$A \triangleq \left. \frac{\partial f}{\partial x} \right|_{(x_e, u_e)}, \quad B \triangleq \left. \frac{\partial f}{\partial u} \right|_{(x_e, u_e)}.$$

The `DynamicalSystem` base class evaluates A, B numerically via central differencing ($\epsilon = 10^{-6}$), while concrete models implement analytical Jacobians.

Numerical Integration. The continuous dynamics are integrated over time steps $h = \Delta t$ using explicit 4th-order Runge–Kutta (RK4):

$$k_1 = f(t_k, x_k, u_k), \quad k_2 = f\left(t_k + \frac{h}{2}, x_k + \frac{h}{2}k_1, u_k\right), \quad k_3 = f\left(t_k + \frac{h}{2}, x_k + \frac{h}{2}k_2, u_k\right), \quad k_4 = f(t_k + h, x_k + hk_3, u_k),$$

$$x_{k+1} = x_k + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4).$$

Holding control input $u(t) \equiv u_k$ constant across each interval $[t_k, t_k + h)$ reflects the physical Zero-Order Hold (ZOH) of digital microcontrollers.

5 Theory: Classical Control — PID Architecture

The continuous-time PID control law with filtered derivative is:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}, \quad e(t) = r(t) - y(t).$$

In discrete time with sampling interval h , derivative action is applied directly to the measured output with low-pass filter time constant $\tau_d = \frac{K_d}{N K_p}$ ($N = 10$) to prevent high-frequency noise amplification:

$$e_k = r_k - y_k, \quad D_k = \frac{\tau_d}{\tau_d + h} D_{k-1} + \frac{K_d}{\tau_d + h} (y_k - y_{k-1}), \quad u_{\text{unsat},k} = K_p e_k + I_k - D_k.$$

Anti-Windup Protection. When actuators reach physical limits ($|u| \leq u_{\text{max}}$), *conditional integration (clamping)* freezes the integrator update ($I_{k+1} = I_k$) whenever u_{unsat} is saturated and e_k has the same sign as u_k , preventing destructive overshoot upon setpoint arrival.

5.1 Worked Example: Stabilising an Unstable Plant (Experiment 03)

We evaluate the controller on an open-loop unstable plant with a Right-Half Plane (RHP) pole:

$$\ddot{y}(t) - \omega_0^2 y(t) = u(t) + d(t), \quad \omega_0 = 2.0 \text{ rad/s}, \quad G(s) = \frac{1}{s^2 - 4}.$$

The system is tested under a unit step reference $r = 1.0 \text{ rad}$, a step input disturbance torque $d = 0.5 \text{ N} \cdot \text{m}$ applied at $t = 5.0 \text{ s}$, and actuator saturation $|u(t)| \leq 10.0 \text{ N} \cdot \text{m}$.

Controller	Rise t_r [s]	Settle t_s [s]	Overshoot M_p	Steady e_{ss}	IAE	ITAE	Energy E_u	Sat. %
P-only ($K_p = 20$)	0.275	10.0	$9.54 \times 10^9\%$	6.04×10^7	4.77×10^7	4.53×10^8	984.3	97.62%
PD ($K_d = 5.66$)	0.389	10.0	28.19%	0.2812	2.811	13.63	316.3	1.60%
PID (no AW)	0.334	6.162	53.08%	3.90×10^{-5}	0.9274	1.043	262.2	5.55%
PID + Anti-Windup	0.362	6.161	39.43%	3.90×10^{-5}	0.7979	0.876	245.1	1.61%

Table 2: Benchmark results for Experiment 03 (regenerated from experiments/03_pid_stabilizes_unstable/table.md).

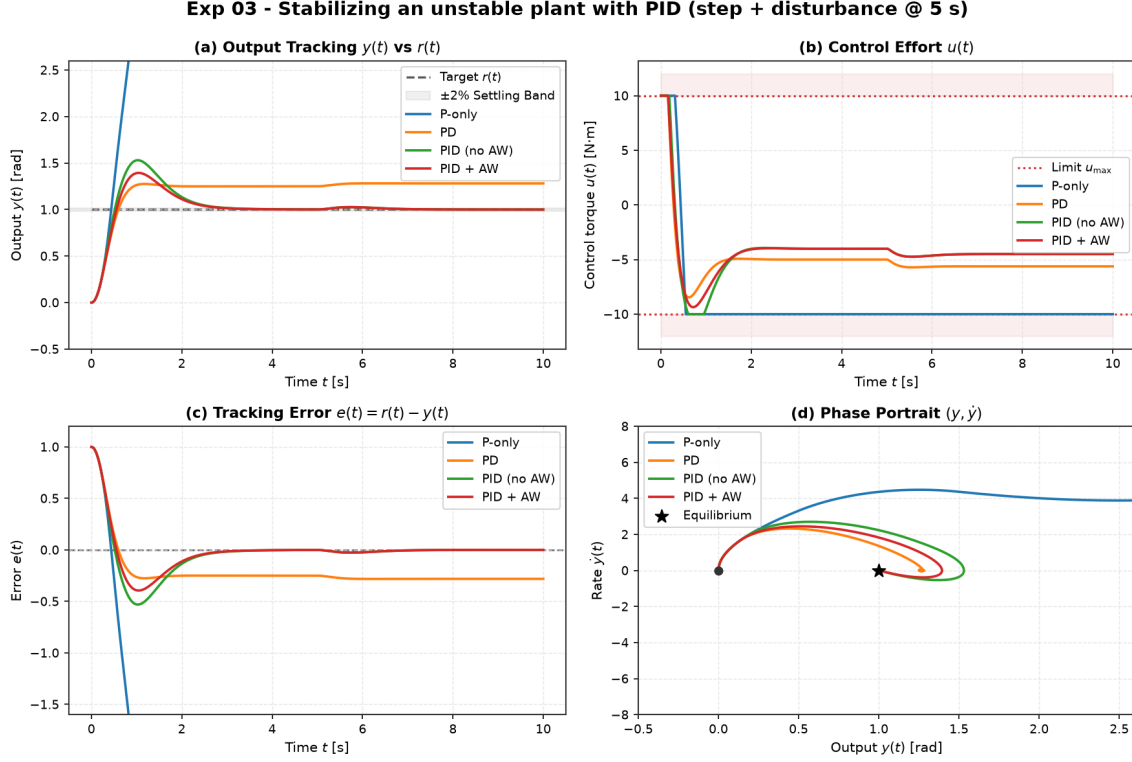


Figure 2: Experiment 03 benchmark comparison. (a) Output tracking $y(t)$, (b) Control effort $u(t)$ with saturation limits $\pm 10 \text{ N} \cdot \text{m}$, (c) Tracking error $e(t)$, and (d) Phase portrait $(y, \dot{y})$.

6 Theory: Modern Control, Estimation, and Optimal Design

6.1 Pole Placement and LQR

For controllable linear systems $\dot{x} = Ax + Bu$, full-state feedback $u = -Kx + k_r r$ assigns arbitrary closed-loop poles $\sigma(A - BK)$. **StateFeedback** computes K via Ackermann's formula:

$$K = \begin{bmatrix} 0 & \dots & 0 & 1 \end{bmatrix} \mathcal{C}(A, B)^{-1} \Delta_{\text{des}}(A), \quad k_r = -\frac{1}{C(A - BK)^{-1}B}.$$

Linear Quadratic Regulator (LQR). Rather than guessing pole locations, LQR computes the optimal feedback gain $K = R^{-1}B^\top P$ minimising the infinite-horizon quadratic cost:

$$J = \int_0^\infty \left(x(t)^\top Q x(t) + u(t)^\top R u(t) \right) dt, \quad Q \succeq 0, \quad R \succ 0.$$

The kernel $P = P^\top \succ 0$ uniquely solves the **Continuous Algebraic Riccati Equation (CARE)**:

$$A^\top P + PA - PBR^{-1}B^\top P + Q = 0.$$

We solve CARE from scratch via the stable invariant subspace of the Hamiltonian matrix:

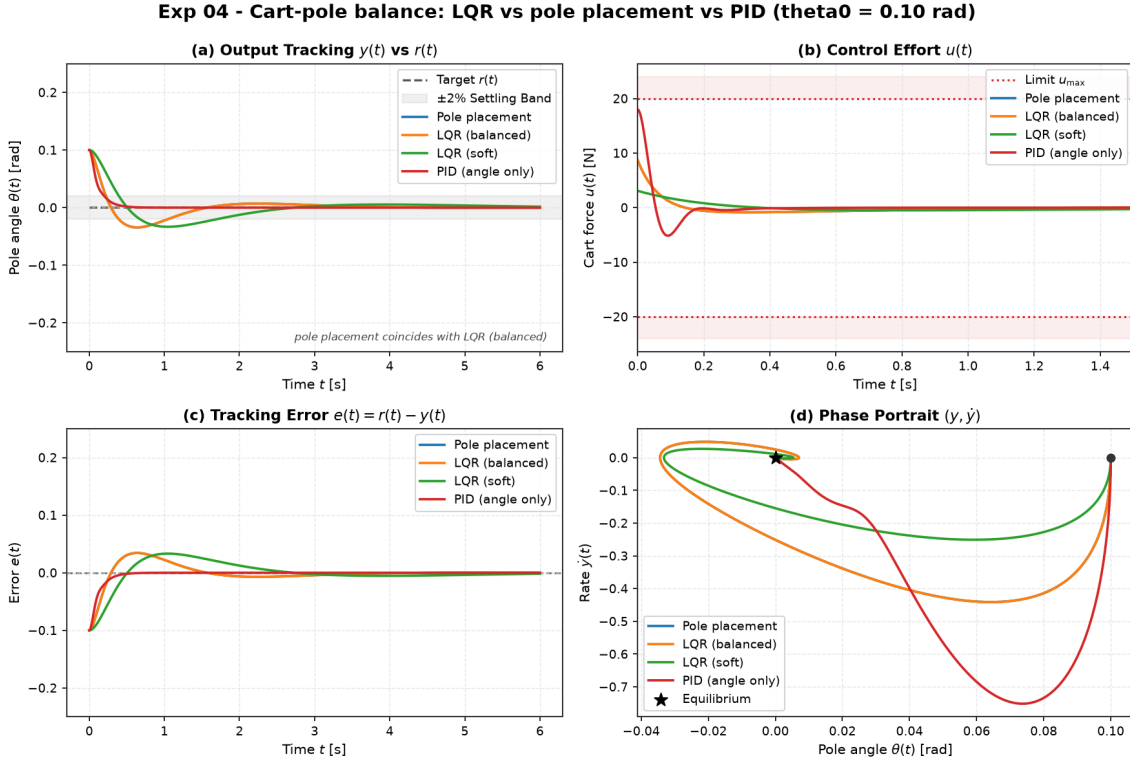
$$\mathcal{H}_{\text{ham}} = \begin{bmatrix} A & -BR^{-1}B^\top \\ -Q & -A^\top \end{bmatrix} \in \mathbb{R}^{2n \times 2n}.$$

Continuous full-state LQR provides guaranteed robustness margins: gain margin $G_m \in [1/2, \infty)$ (-6 dB to $+\infty$ dB) and phase margin $\Phi_m \geq 60^\circ$ in all channels.

6.2 Worked Example: Cart-Pole Regulation (Experiment 04)

Controller	Settle θ t_s [s]	RMSE θ [rad]	Energy E_u [N^2s]	Peak $ u _{\max}$ [N]	Peak $ x _{\max}$ [m]	Status
Pole Placement	1.05	0.0171	2.34	8.71	0.162	Stable
LQR (Balanced)	1.05	0.0171	2.34	8.71	0.162	Stable
LQR (Soft)	1.70	0.0223	0.959	3.08	0.321	Stable
PID (Angle Only)	0.20	0.0110	8.76	18.00	0.823	Stable (Cart Drifts)

Table 3: Benchmark results for Experiment 04 on nonlinear Cart-Pole (from experiments/04_lqr_vs_pole_placement_cartpole/table.md).



6.3 State Estimation: Observers, Kalman Filtering, and Duality

Mathematical Duality: LQR Control vs. Kalman Filtering (LQE).

LQR Regulator (CARE): $A^\top P + PA - PBR^{-1}B^\top P + Q = 0, \quad K = R^{-1}B^\top P$

LQE Estimator (FARE): $AP + PA^\top - PC^\top R_v^{-1}CP + Q_w = 0, \quad L = PC^\top R_v^{-1}$

6.4 Worked Example: LQG on the Cart-Pole (Experiment 06)

Controller	Settle θ t_s [s]	RMSE θ [rad]	Energy E_u [N ² s]	Peak $ u _{\max}$ [N]	Status
LQR (Full State, Clean)	0.942	0.0150	1.49	6.97	Ideal Baseline
LQG (Optimal Filter)	2.070	0.0249	2.21	3.28	Optimal Smooth
Luenberger + K	0.496	0.0165	16.20	15.10	Severe Noise Chattering
LQG (Overconfident $V/100$)	0.828	0.0215	2.97	4.80	Noise Amplified

Table 4: Benchmark results for Experiment 06 comparing LQG, Luenberger observer, and clean LQR under measurement noise (from experiments/06_lqg_vs_lqr_measurement_noise/table.md).

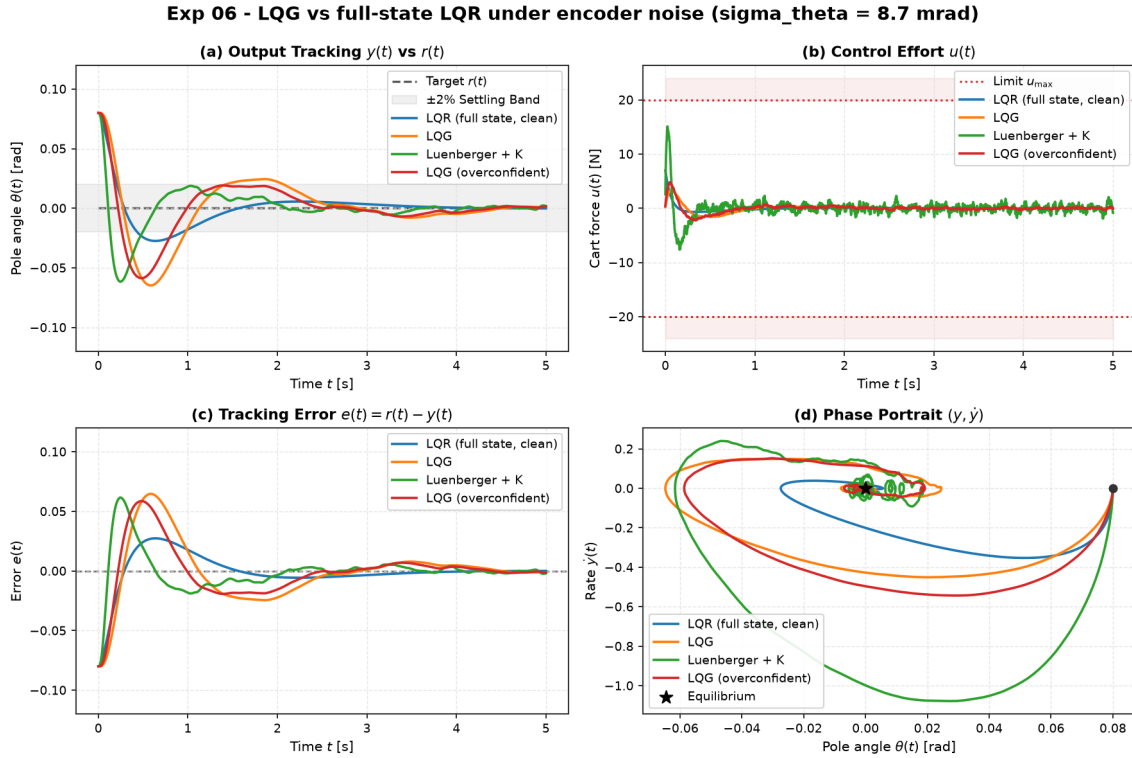
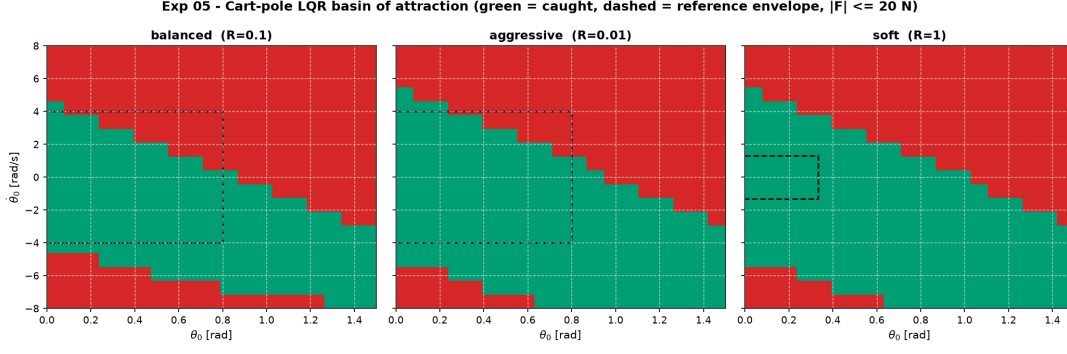


Figure 4: Experiment 06 benchmark comparison under encoder noise.

6.5 Nonlinear Basin of Attraction: Cart-Pole LQR Robustness (Experiment 05)

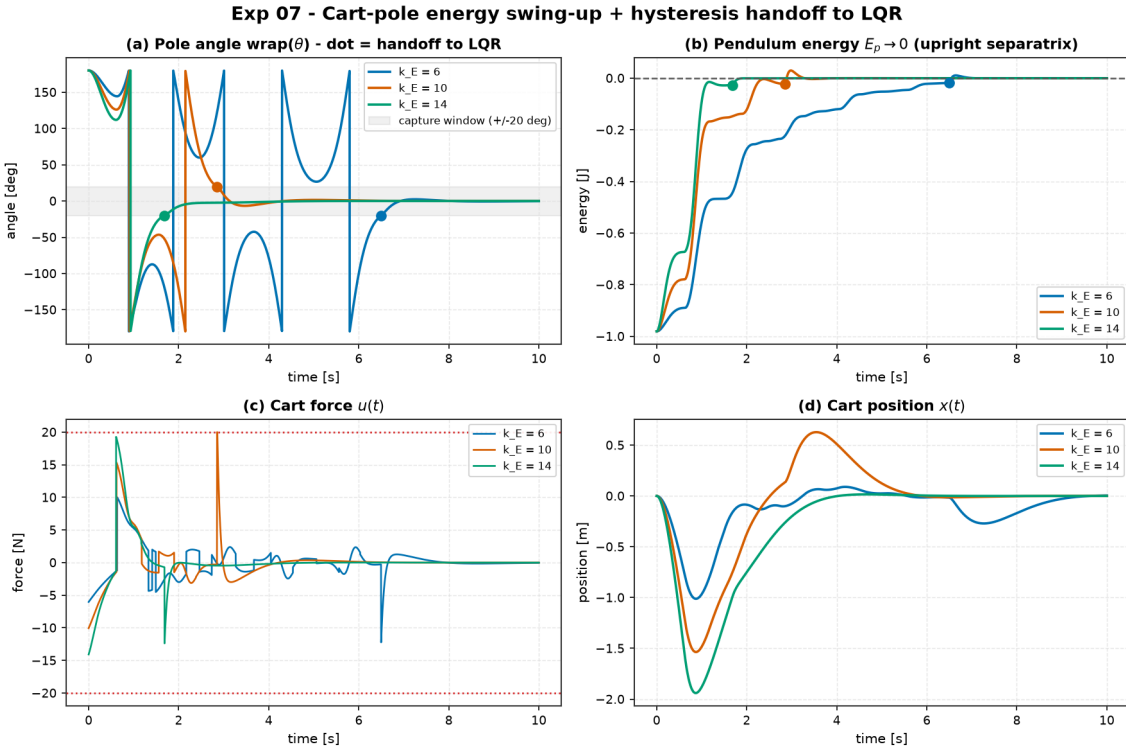
Tuning	Effort R	Measured θ_0 Edge	Lyapunov Est.	Measured $\dot{\theta}_0$ Edge	Lyapunov Est.
Balanced	0.10	0.83 rad (48°)	0.80 rad (46°)	4.4 rad/s	4.0 rad/s
Aggressive	0.01	0.92 rad (53°)	0.80 rad (46°)	5.3 rad/s	4.0 rad/s
Soft	1.00	1.00 rad (57°)	0.33 rad (19°)	5.3 rad/s	1.3 rad/s

Table 5: Measured recoverable boundaries vs. theoretical Lyapunov sub-level sets on nonlinear Cart-Pole.

Figure 5: Experiment 05 2D ($\theta_0 \times \dot{\theta}_0$) basin of attraction maps.

6.6 Nonlinear Control & Hybrid Swing-Up: Spong Energy Shaping (Experiment 07)

Energy Gain k_E	Capture t_{cap} [s]	Settle t_s [s]	Switches	Energy E_u [N ² s]	Peak $ u _{\text{max}}$ [N]	Cart Excursion [m]	Status
$k_E = 6$	6.496	7.452	1	52.28	12.23	1.012	Balanced
$k_E = 10$	2.852	4.058	1	87.28	20.00	1.536	Balanced
$k_E = 14$	1.686	3.344	1	107.24	19.27	1.940	Balanced

Table 6: Benchmark results for Experiment 07 swing-up from $\theta_0 = \pi$ across energy gains $k_E \in \{6, 10, 14\}$ under $|F| \leq 20$ N (from experiments/07_cartpole_swingup_hybrid/table.md).Figure 6: Experiment 07 trajectory traces during swing-up from hanging ($\theta_0 = \pi$).

6.7 System Identification & Data-Driven Control (Experiment 09)

Data Length	Relative Error A	Relative Error K_{id}	RMSE θ [rad]	Stable Runs	Status
300 steps (1 s)	5.86×10^0	0.992	26.77	1 / 6	Severe Model Drift / Instability
1500 steps (3 s)	4.19×10^{-1}	0.527	0.0336	4 / 6	Marginal / Oscillatory
12000 steps (24 s)	2.17×10^{-1}	0.493	0.0246	6 / 6 (100%)	Consistently Stable

Table 7: Experiment 09 Monte Carlo evaluation (6 seeds) of LQR designed on identified models (from experiments/09_control_on_identified_model/table.md).

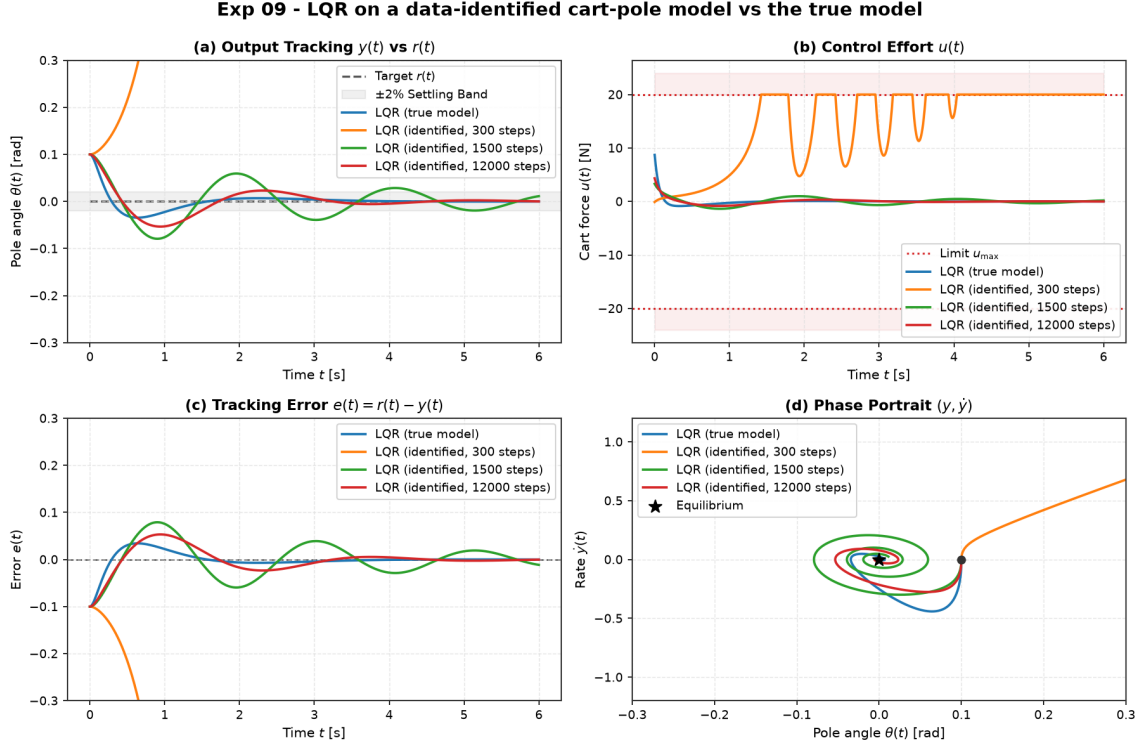


Figure 7: Experiment 09 benchmark comparison showing LQR performance designed on identified models.

6.8 Constrained Model Predictive Control (Experiment 08)

Controller	Cart Peak $ x _{\max}$ [m]	Constraint Violation [m]	Settle θ t_s [s]	Energy E_u [N ² s]	Status
LQR	0.6162	0.1162 (23.2%)	2.79	28.8	Rail Violation
MPC (Unconstrained)	0.6232	0.1232 (24.6%)	2.81	27.7	Rail Violation
MPC ($ x_{\text{cart}} \leq 0.5$ m)	0.5014	0.0014 (0.28%)	2.56	46.3	Strictly Compliant

Table 8: Benchmark results for Experiment 08 on nonlinear Cart-Pole with $|F| \leq 20$ N and rail limit $|x| \leq 0.5$ m from $\theta_0 = 0.35$ rad (from experiments/08_mpc_vs_lqr_constrained_cartpole/table.md).

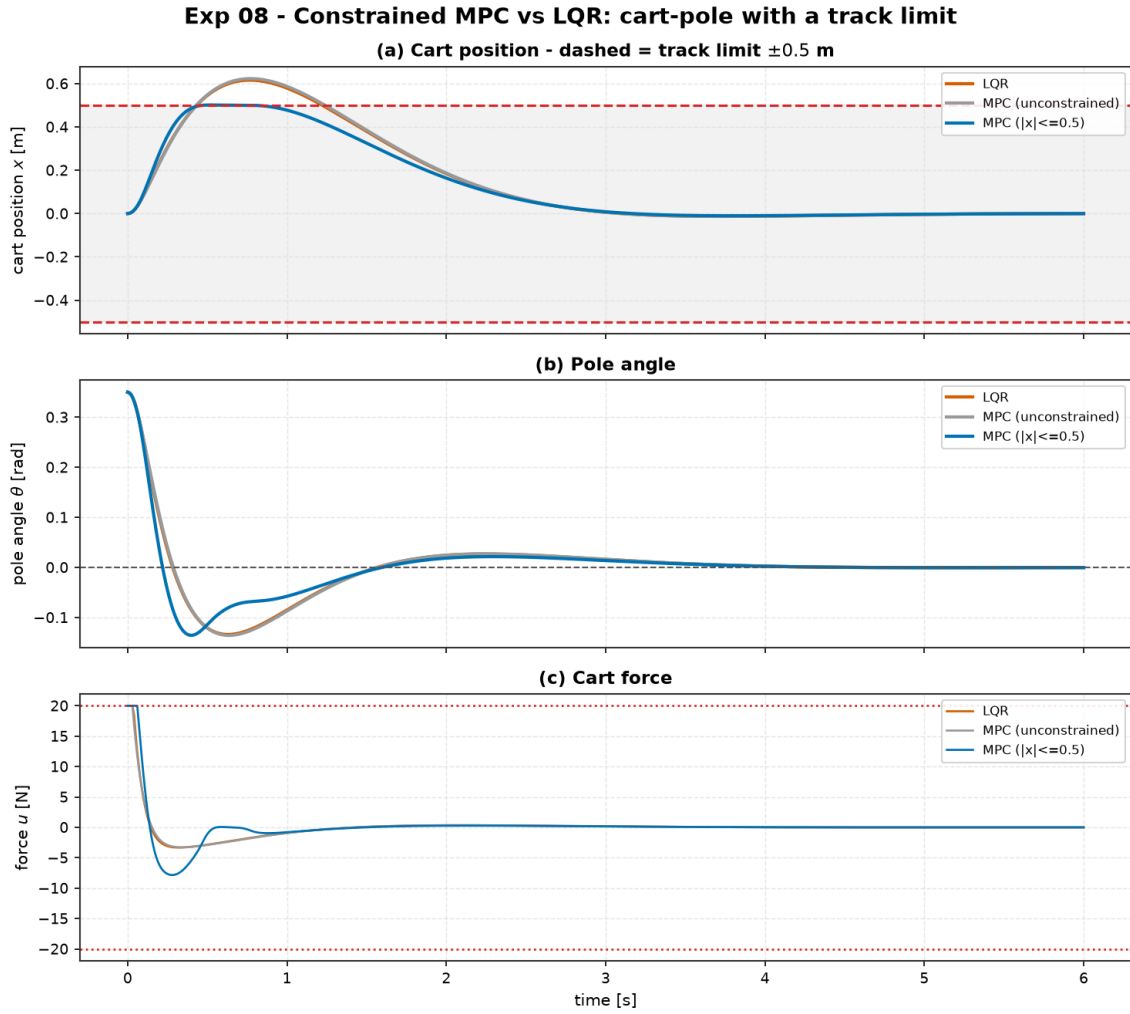


Figure 8: Experiment 08 trajectory traces highlighting Cart Position $x(t)$ with the ± 0.5 m track limit.

7 Theory: Machine Learning and Learned Dynamics (Module 06)

7.1 Worked Example: Planning with Learned Dynamics vs. True Model (Experiment 10)

Controller	Settle θ t_s [s]	RMSE θ [rad]	Energy E_u [N^2s]	Peak $ u _{\max}$ [N]	Status
LQR (True Model)	1.26	0.0374	8.42	17.40	Analytic Optimum
SamplingMPC (True Model)	4.44	0.0448	10.80	6.59	Stable (CEM True)
SamplingMPC (Learned Model)	3.56	0.0462	10.20	6.00	Stable (CEM Learned)

Table 9: Benchmark results for Experiment 10 comparing CEM-MPC on learned neural dynamics vs. true model and LQR (from experiments/10_planning_learned_vs_true_model/table.md).

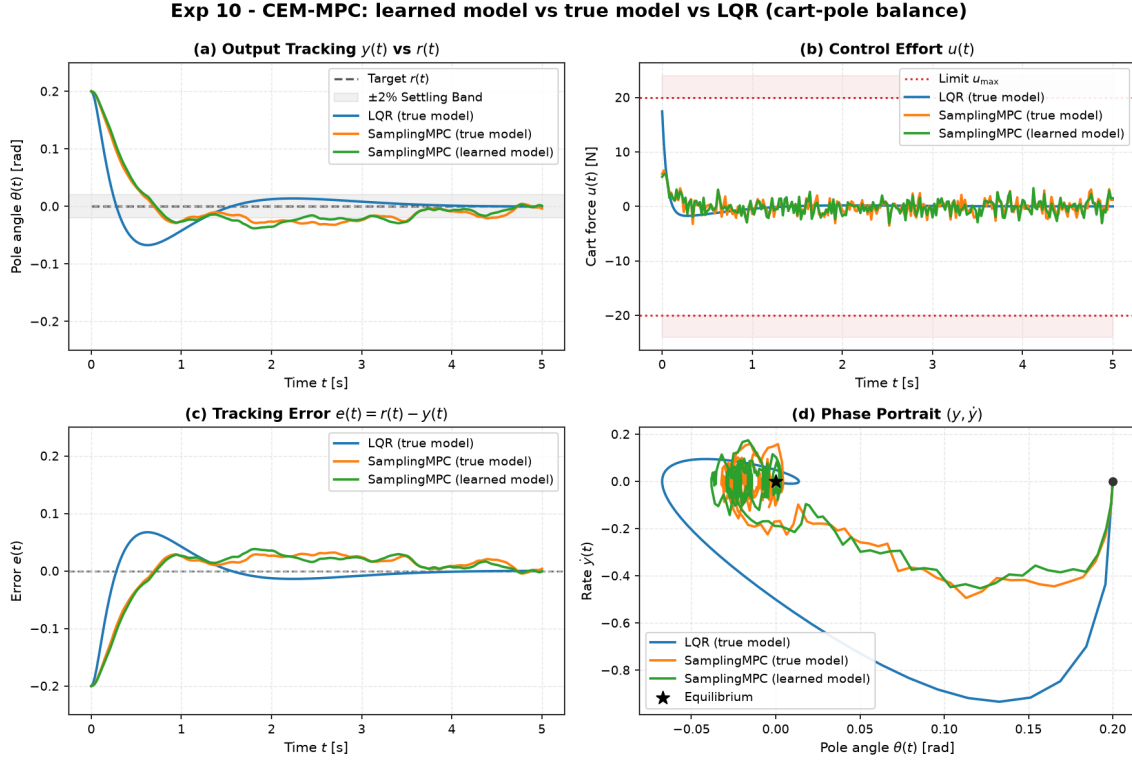


Figure 9: Experiment 10 benchmark comparison: SamplingMPC on Learned Neural Model vs. True Model.

8 Theory: Reinforcement Learning for Dynamical Systems (Module 07)

8.1 Worked Example: Tabular Q-Learning vs. Classical Control on Pendulum (Experiment 11)

Controller	Task	Min Err [rad]	Held Upright?	t_{upright} [s]	Final Err [rad]	Energy E_u	Train Samples	Parameters
Energy-Shaping + LQR	Swing-Up	0.00	Yes	4.00	0.000	45.9	0	3
Tabular Q-Learning	Swing-Up	0.02	No	9.10	1.479	101.2	450,000	61,875
LQR	Balance	0.00	Yes	0.85	0.000	11.7	0	2
Tabular Q-Learning	Balance	0.05	No	3.45	1.527	40.4	180,000	51,975

Table 10: Benchmark results for Experiment 11 comparing Tabular Q-Learning against Classical Energy-Shaping and LQR on the Inverted Pendulum (from experiments/11_qlearning_vs_classical/table.md).

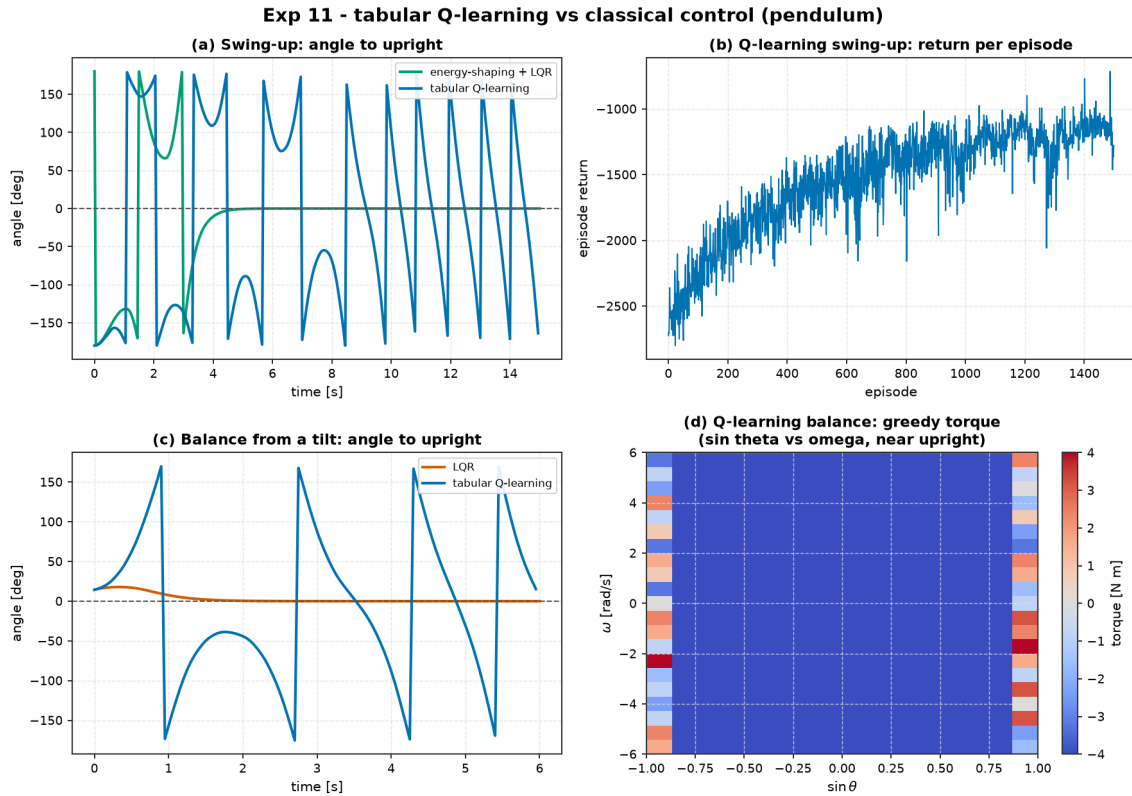


Figure 10: Experiment 11 benchmark comparison: Tabular Q-Learning vs. Classical Control on Inverted Pendulum.

8.2 Worked Example: The RL Zoo vs. LQR on Cart-Pole Balance (Experiment 18)

Agent / Controller	Greedy Return	Held Steps (/200)	Env Steps	Train Time [s]	Parameters
LQR (Analytic CARE)	-0.3	200.0	0	0.0	4
Tabular Q-Learning	-55.5	92.0	300,000	7.1	354,375
DQN (From Scratch)	-2.6	200.0	90,177	68.7	4,805
PPO (From Scratch)	-0.3	200.0	240,000	72.6	4,545
PPO (Stable-Baselines3)	-44.9	52.5	120,000	129.6	9,091

Table 11: Benchmark results for Experiment 18 comparing the RL zoo against LQR on Cart-Pole balance (from `experiments/18_rl_zoo_vs_lqr/table.md`).

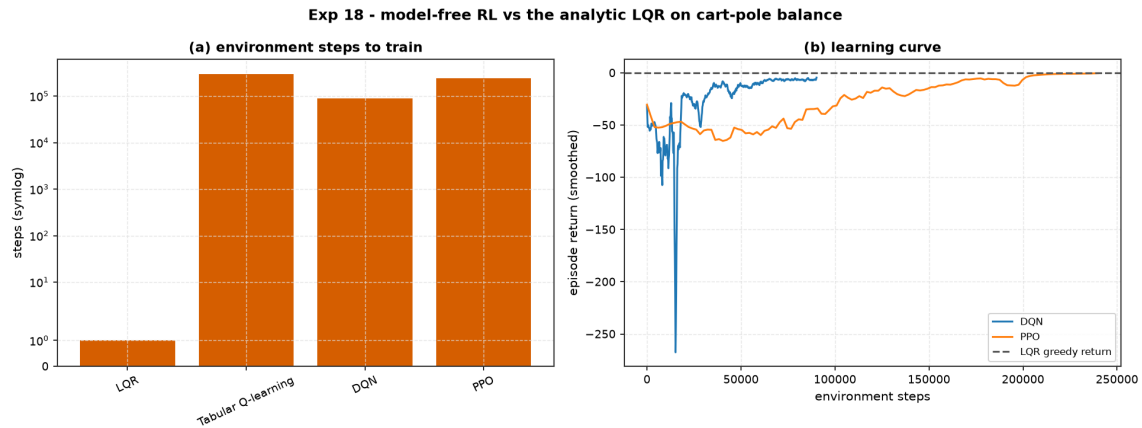


Figure 11: Experiment 18 benchmark comparison: RL Zoo vs. LQR on Cart-Pole balance.

9 Theory: AI + Control Hybrid Architectures (Module 08)

9.1 Worked Example: Shielded Tabular Q-Learning on Pendulum (Experiment 12)

Controller	Min Err [rad]	Final Err [rad]	Held Upright?	t_{upright} [s]	Energy E_u	Shield Active
Tabular Q-Learning (Raw)	0.028	1.406	No	10.30	105.4	0.0%
Shielded (RL + Classical Finisher)	0.000	0.000	Yes	9.30	68.0	43.7%

Table 12: Benchmark results for Experiment 12 comparing Raw Tabular Q-Learning vs. Shielded Hybrid Control on the Inverted Pendulum (from experiments/12_shielded_qlearning/table.md).

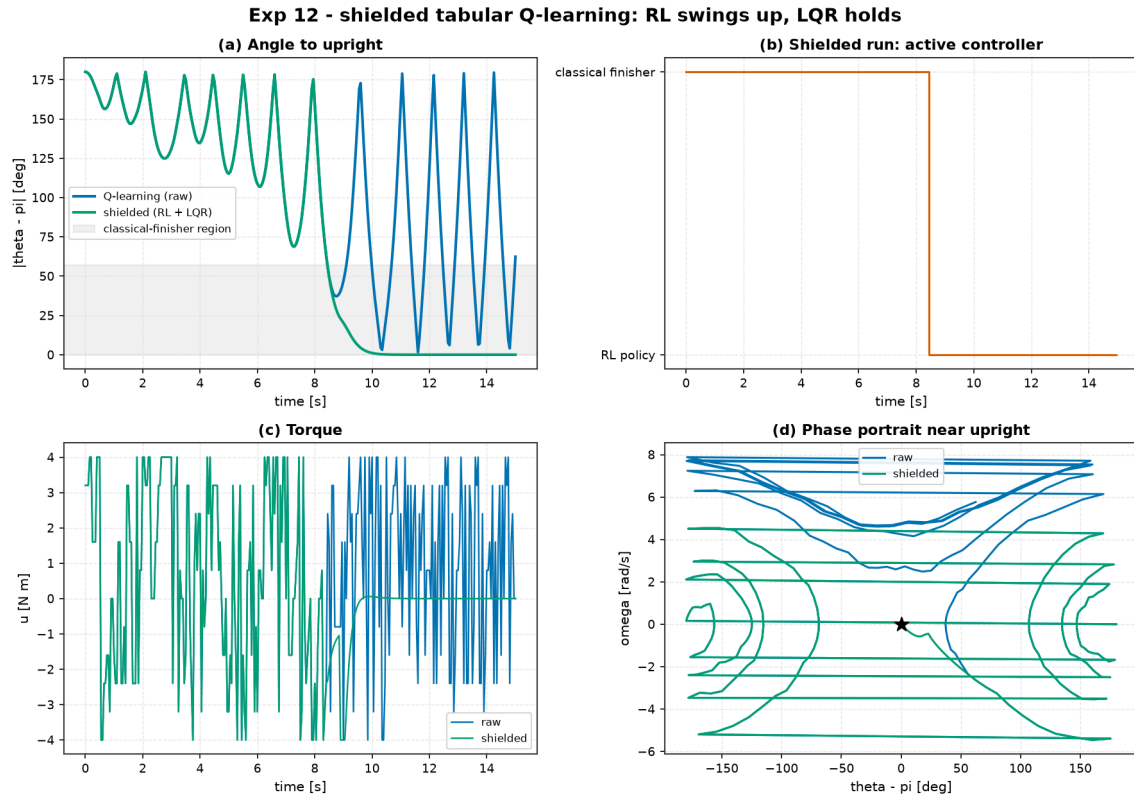


Figure 12: Experiment 12 benchmark comparison: Raw Tabular Q-Learning vs. Shielded Hybrid Control on Inverted Pendulum.

10 Theory: Advanced State Estimation, Adaptive Control, and Robotics Capstones (Module 09)

10.1 Worked Example: Crazyflie 2.0 Flying a Lemniscate (Experiment 14)

Controller	RMS Pos Err [mm]	Max Pos Err [mm]	RMS Pitch [°]	Control Energy $\int \ \Delta u\ ^2$	Thrust Saturation
LQR + Flatness Feedforward	43.5	112.0	4.3°	0.033	0.7%
LQR Feedback Only	51.0	109.0	4.4°	0.038	0.7%
Linear MPC (Single Setpoint)	142.0	208.0	4.3°	0.026	0.1%
Linear MPC (Reference Preview)	47.9	134.0	4.5°	0.033	0.6%

Table 13: Benchmark results for Experiment 14: Crazyflie 2.0 figure-8 tracking under 11% wind gust (from `experiments/14_quadrotor_figure8_tracking/table.md`).

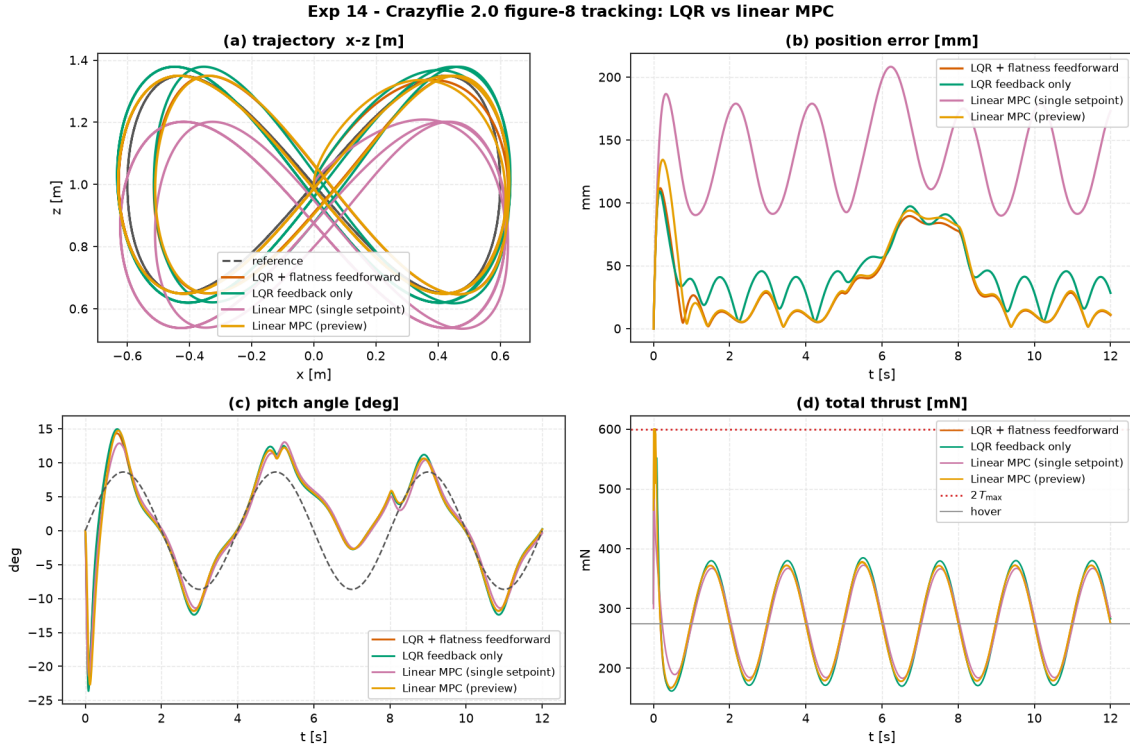


Figure 13: Experiment 14 benchmark comparison: Crazyflie 2.0 planar quadrotor flying a lemniscate figure-8 through a sustained wind gust.

10.2 Worked Example: EKF Output Feedback under Noisy Partial Sensing (Experiment 15)

Controller	RMS Pos Err [mm]	Max Pos Err [mm]	RMS Pitch [°]	Control Energy $\int \ \Delta u\ ^2$	RMS Vel Err [m/s]
LQR (Clean Full State)	9.04	49.0	1.26°	0.0024	—
LQR + EKF (Nonlinear Observer)	9.51	50.6	1.25°	0.0026	0.008
LQR + Finite-Difference Vel	18.32	47.4	1.89°	0.3580 (150×)	0.332 (40×)

Table 14: Benchmark results for Experiment 15: Crazyflie 2.0 output feedback tracking with EKF vs. naive differencing (from experiments/15_quadrotor_ekf_output_feedback/table.md).

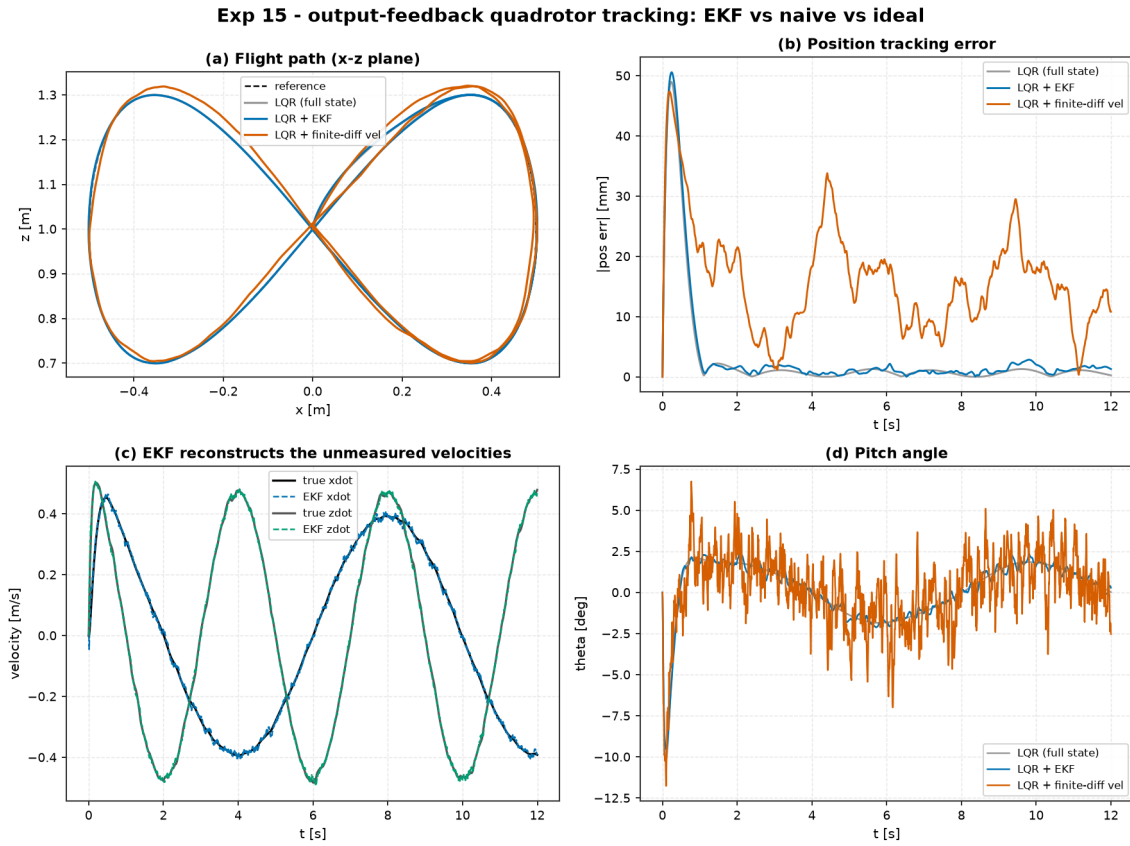


Figure 14: Experiment 15 benchmark comparison: EKF Output-Feedback Quadrotor Tracking.

10.3 Worked Example: EKF vs. UKF on Nonlinear Pendulum Estimation (Experiment 16)

Regime	Filter	RMS Error [rad]	Final Error [rad]	Recovered?
Hard (sin/cos, π -off init)	EKF	6.514	6.282 ($\approx 2\pi$)	False (Trapped)
Hard (sin/cos, π -off init)	UKF	1.382	0.068	True (Converged)
Mild (sin/rate, good init)	EKF	0.021	0.026	True
Mild (sin/rate, good init)	UKF	0.021	0.026	True

Table 15: Benchmark results for Experiment 16: EKF vs. UKF on nonlinear Pendulum estimation (from `experiments/16_ekf_vs_ukf/table.md`).

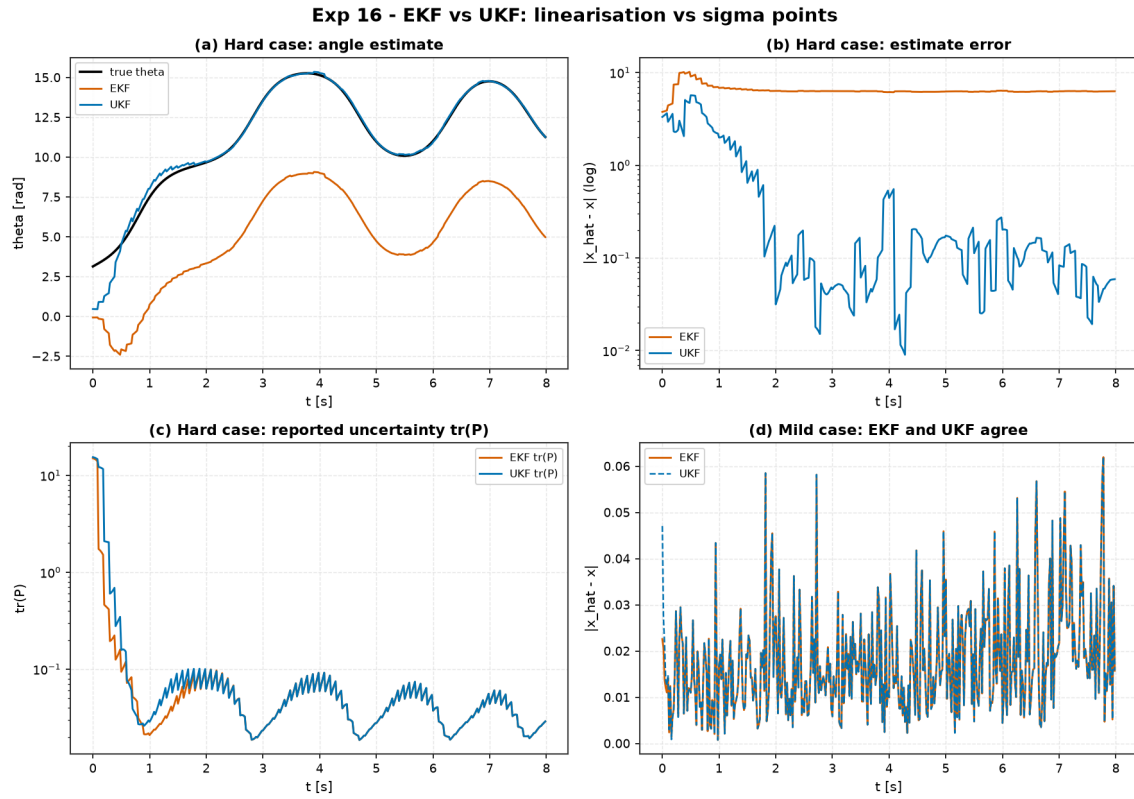


Figure 15: Experiment 16 benchmark comparison: EKF vs. UKF on nonlinear Pendulum estimation.

10.4 Worked Example: Adaptive vs. Fixed Control on a Drifting Plant (Experiment 17)

Controller	RMS Err (Early $t \leq 5\text{s}$)	RMS Err (Late $t \geq 15\text{s}$)	Final Settle Err [m]	Control Energy E_u
LQR (Nominal $k = 1$)	0.2573	0.4013	0.4217	180.7
LQR (Worst-Case $k = 5$)	0.3577	0.5194	0.5406	123.3
MRAC (Adaptive)	0.0008	0.0008	0.0008	413.5
GainScheduled LQR	0.2916	0.5075	0.5396	142.3

Table 16: Benchmark results for Experiment 17: Adaptive MRAC vs. Fixed LQR on a drifting plant (from `experiments/17_adaptive_vs_fixed_changing_plant/table.md`).

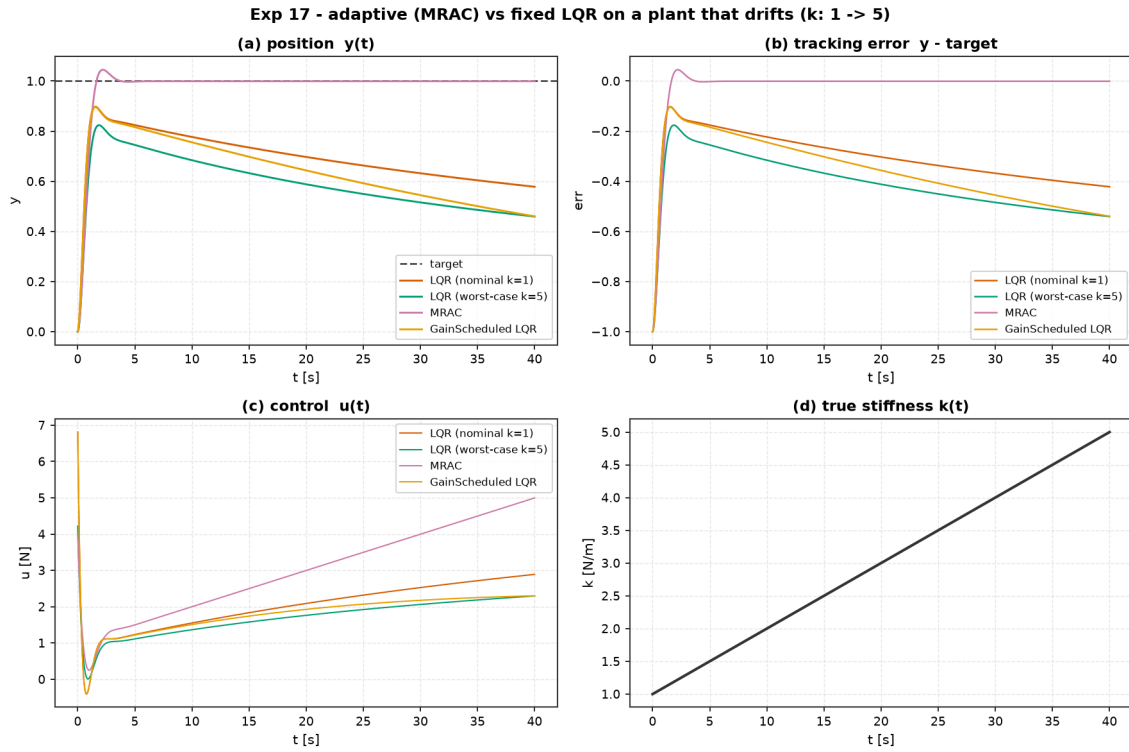


Figure 16: Experiment 17 benchmark comparison: Adaptive MRAC vs. Fixed LQR on drifting spring plant.

10.5 Worked Example: Obstacle-Aware Nonlinear MPC on the Quadrotor (Experiment 20)

Controller	RMS Pos Err [mm]	Min Clearance [mm]	Steps in Keep-Out	Control Energy E_u
LQR + Flatness Feedforward	86.26	-85.80	26 (Violates)	0.4804
SamplingMPC (True Dynamics)	234.4	+10.88	0 (Compliant)	0.0329
SamplingMPC (Learned Grey-Box Model)	254.6	+26.06	0 (Compliant)	0.0343

Table 17: Benchmark results for Experiment 20: Obstacle-aware nonlinear MPC on Crazyfly 2.0 (from `experiments/20_quadrotor_obstacle_nmpc/table.md`).

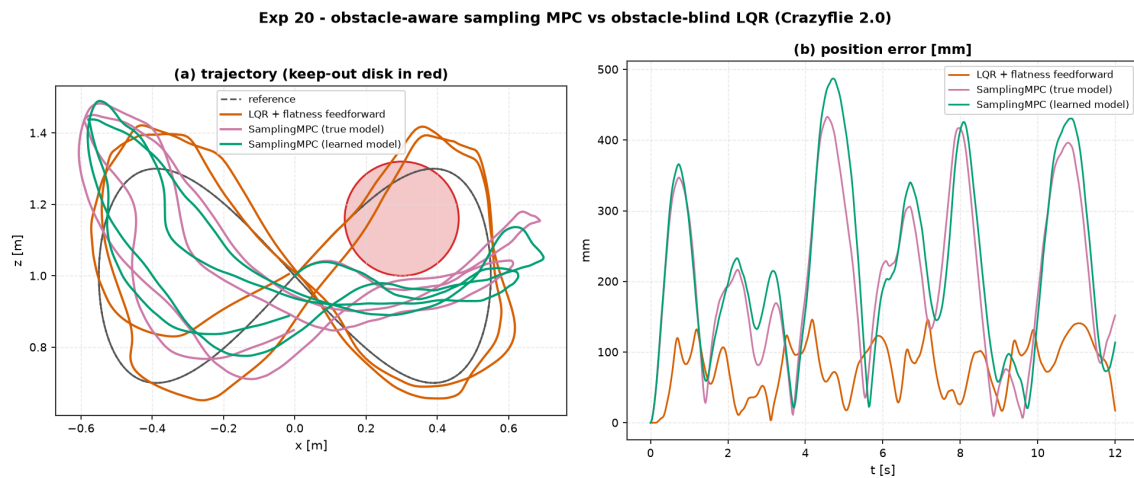


Figure 17: Experiment 20 benchmark comparison: Obstacle-aware nonlinear MPC on Crazyfly 2.0 quadrotor.

10.6 Worked Example: The Grand Capstone Bake-Off (Experiment 21)

Experiment 21 conducts the ultimate five-way capstone bake-off on the full Crazyflie 2.0 figure-8 course combined with a keep-out obstacle disk ($r = 0.16$ m at $(0.30, 1.16)$ m) and a 30 mN lateral wind gust during $t \in [4.5, 7.5]$ s. We evaluate every major paradigm under the standardized multi-objective scoring rubric (tracking RMSE 0.40, energy 0.20, slew 0.10, safety compliance 0.15, wind robustness 0.15):

Controller Paradigm	RMS [mm]	Robust RMS [mm]	Energy E_u	Slew	Obstacle Steps	Thrust Sat.	Latency [ms]	Score
LQR + Flatness	99.9	102.0	0.483	378.0	28 (DQ)	72.4%	0.05	0.0
Linear MPC (Preview)	243.0	242.0	0.0034	0.0639	20 (DQ)	0.0%	11.5	0.0
Sampling MPC (Learned)	316.0	275.0	0.0334	14.3	0 (Clean)	0.0%	38.4	8.0 (PASS)
Imitation Policy (BC+PPO)	41.4	35.7	0.0040	0.0580	46 (DQ)	0.0%	0.09	0.0
Imitation + MPC Shield	317.0	283.0	0.0316	12.8	0 (Clean)	0.0%	38.4	7.9 (PASS)

Table 18: Benchmark results for Experiment 21: The Grand Capstone Bake-Off on Crazyflie 2.0 (from `experiments/21_grand_capstone_bakeoff/table.md`).

Key Capstone & Retrospective Takeaways:

1. **Only Obstacle-Aware Planners Complete the Course:** Sampling MPC (learned grey-box model) is the sole entry with zero keep-out violations and zero saturation, achieving the highest score on the board (8.0).
2. **The Best Tracker Cannot Be Trusted:** The imitation policy (behavioral cloning from LQR + PPO fine-tuning) achieves the lowest nominal tracking error (41.4 mm), but cuts straight through the keep-out disk 46 times and lacks wind rejection. Pure RL from scratch cannot bootstrap on the tiny nano-drone inertia ($I_{yy} = 1.4 \times 10^{-5}$ kg·m²), proving the necessity of expert demonstrations.
3. **Does Combining Help? Barely Here, and That is the Honest Finding:** The hybrid "Imitation + MPC shield" completes the course cleanly (0 violations, score 7.9), but only ties sampling MPC alone (8.0) because the shield spends most of its time in the MPC fallback, diluting the policy's latency advantage (0.09 ms vs 38.4 ms).
4. **No Universal Winner Across Engineering Trade-Offs:** Classical flatness is ultra-fast (0.05 ms) but obstacle-blind; MPC respects non-convex constraints but exceeds the 20 ms flight-controller deadline (38.4 ms); imitation is instant but replicates expert blind spots.

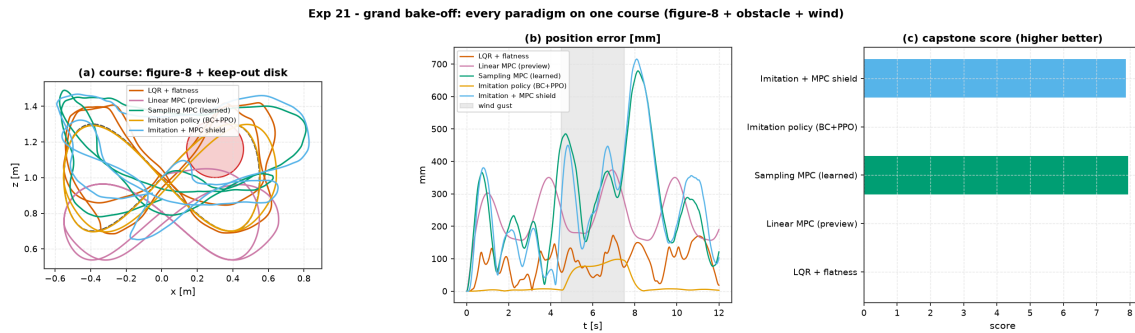


Figure 18: Experiment 21 Grand Capstone Bake-Off on Crazyflie 2.0 quadrotor: Course layout, position error over time, and multi-objective score breakdown.

11 Theory: The Intelligent Control Challenge (ICC) Benchmark (Module 10)

11.1 Worked Example: Multi-Paradigm Challenge Leaderboard (Experiment 19)

Submission Paradigm	Track 1 (MSD)	Track 1 (DC Motor)	Track 2 (Pendulum)	Track 2 (Cart-Pole)	Track 3 (Robustness)
Linear MPC	21.1 (PASS)	41.3 (PASS)	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)
LQR	7.4 (PASS)	36.8 (PASS)	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)
Energy+LQR Hybrid	1.8 (PASS)	0.0 (FAILED)	23.8 (PASS)	9.6 (PASS)	29.9 (PASS)
PID	0.6 (PASS)	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)
Tabular Q-Learning	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)	0.0 (FAILED)

Table 19: Experiment 19 Intelligent Control Challenge Leaderboard: Composite Score / 100 (from `experiments/19_icc_leaderboard/leaderboard.md`).

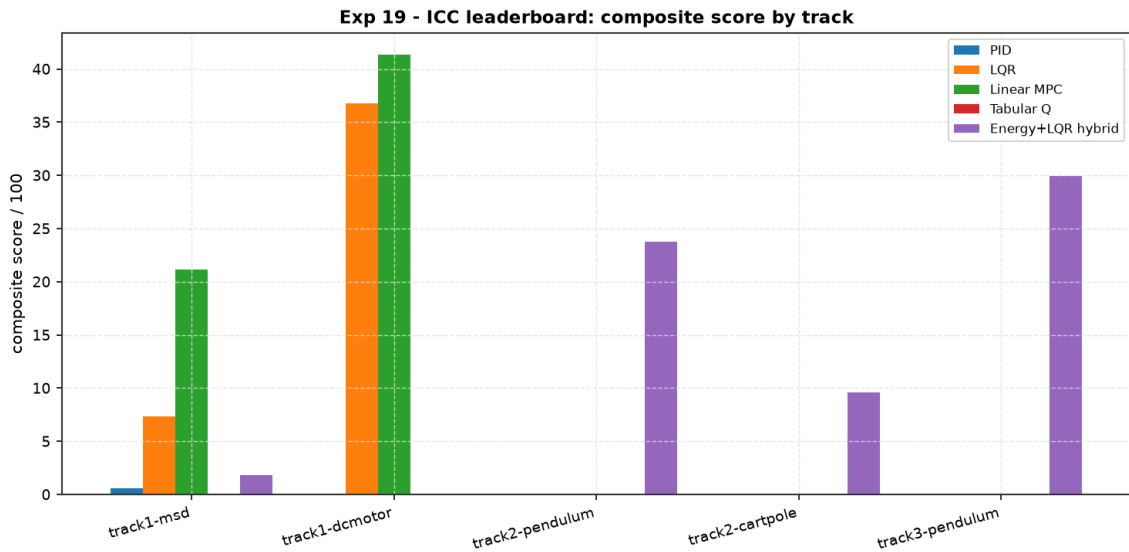


Figure 19: Experiment 19 Intelligent Control Challenge Leaderboard: Composite score comparison across all five paradigms and benchmark tracks.

12 Project Status and Roadmap

Milestone M11 — The Grand Capstone & Retrospective — Completed. All curriculum modules, capstone robotics showcases, deep RL agents, adaptive controllers, and benchmark challenge suites are fully implemented from scratch, tested, and validated across 21 experiments:

- Module 01: Mathematical Foundations** — Linear algebra, matrix exponential, Runge-Kutta integration, numerical optimization.
- Module 02: Dynamic System Modeling** — First-principles modeling, state-space representations, Jacobian linearisation, discretization.
- Module 03: Classical Control** — Filtered derivative PID, conditional anti-windup clamping, frequency response, stability margins (Experiment 03).
- Module 04: Modern Control & Estimation** — Controllability, observability, Ackermann pole placement, full-state Luenberger observers, Filter ARE duality, continuous/discrete Kalman filtering with Joseph update, Extended Kalman Filter (EKF), Unscented Kalman Filter (UKF) (Experiments 04, 05, 06, 15, 16).
- Module 05: Optimal & Constrained Control** — Continuous Algebraic Riccati Equation (CARE) Hamiltonian solver, LQR robustness margins, constrained Model Predictive

Control with active-set QP solver and reference preview (Experiments 08, 14).

6. **Module 06: System ID & Machine Learning** — Least-squares state-space ID, DMDc, matrix logarithm ZOH inversion, from-scratch MLP with Adam, residual/grey-box LearnedDynamics, SamplingMPC (CEM) with CARE terminal cost (Experiments 07, 09, 10, 20).
7. **Module 07: Reinforcement Learning** — Gymnasium ControlEnv adapter, state/action discretization, Tabular Q-Learning, DQN, REINFORCE, and PPO (Experiments 11, 18).
8. **Module 08: AI + Control (Hybrid Architectures)** — Supervisory safety shielding, action filtering, barrier predicates, auditable intervention logging (Experiment 12).
9. **Module 09: Advanced State Estimation, Adaptive Control & Robotics** — Full 6-state Planar Quadrotor (Crazyflie 2.0), differential flatness feedforward trajectory generation, Bryson-rule scaling, EKF/UKF output feedback, Model Reference Adaptive Control (MRAC), and obstacle-aware Sampling NMPC (Experiments 14, 15, 16, 17, 20).
10. **Module 10: The Intelligent Control Challenge & Grand Synthesis** — Standardized multi-track benchmarking engine, robustness sweeps, cross-paradigm leaderboard, and 5-way capstone grand bake-off (Experiments 19, 21).

CI passes on Python 3.10–3.13 with **314 passing unit tests**.

Reproduction Commands.

```
git clone https://github.com/zaliththomas-ui/ai-meets-control-theory.git
cd ai-meets-control-theory
pip install -e ".[dev,ml,xcheck]"
python -m pytest -q
python experiments/03_pid_stabilizes_unstable/run.py
python experiments/04_lqr_vs_pole_placement_cartpole/run.py
python experiments/05_cartpole_basin_of_attraction/run.py
python experiments/06_lqg_vs_lqr_measurement_noise/run.py
python experiments/07_cartpole_swingup_hybrid/run.py
python experiments/08_mpc_vs_lqr_constrained_cartpole/run.py
python experiments/09_control_on_identified_model/run.py
python experiments/10_planning_learned_vs_true_model/run.py
python experiments/11_qlearning_vs_classical/run.py
python experiments/12_shielded_qlearning/run.py
python experiments/14_quadrotor_figure8_tracking/run.py
python experiments/15_quadrotor_ekf_output_feedback/run.py
python experiments/16_ekf_vs_ukf/run.py
python experiments/17_adaptive_vs_fixed_changing_plant/run.py
python experiments/18_rl_zoo_vs_lqr/run.py
python experiments/19_icc_leaderboard/run.py
python experiments/20_quadrotor_obstacle_nmpc/run.py
python experiments/21_grand_capstone_bakeoff/run.py
```